Divide and Conquer: Counting Inversion

(Algorithm Design and Analysis (ADA) - CSE222)

Diptapriyo Majumdar

January 23, 2022 23

Merge Sort

Input: An array A of n numbers (indexed from $1, \ldots, n$). Input size is n.

- Divide the array into $A_{\text{left}} = A[1, \ldots, \lceil n/2 \rceil]$ and $A_{\text{right}} = A[\lceil n/2 \rceil + 1, \ldots, n]$. (Divide step) → creates subproblems
- Recursively sort A_{left} and Recursively sort A_{right}. (Conquer step) two subproblem
- Merge two sorted arrays A_{left} and A_{right} into a sorted array. (Combine step)

Assumptions

Some Assumptions:

Big-Oh

- Addition of two numbers can be performed in $\mathcal{O}(1)$-time.
- Comparison between two numbers can be done in $\mathcal{O}(1)$-time

Divide and Conquer

Subproblem: The same problem but of smaller size

- **Divide Step:** Divide the instance into smaller sized instances, e.g. dividing an input instance of size n into three parts, each of size $n/3$, dividing an input instance of size n into two parts, one of size $3n/5$ and the other part is of size $2n/5$.
- **Conquer Step:** Recursively solve the problem for each parts of the input. subproblem solving
- **Combine Step:** Combine the solutions of smaller parts into a solution for the original instance.

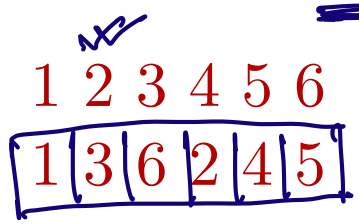
Outline

Comparison of two different rankings

- How similar are rankings given by two different people?
- You rank n distinct songs.
- A collaborative filtering system consults it's database and find another person with similar rankings.
- How do we measure this similarity?

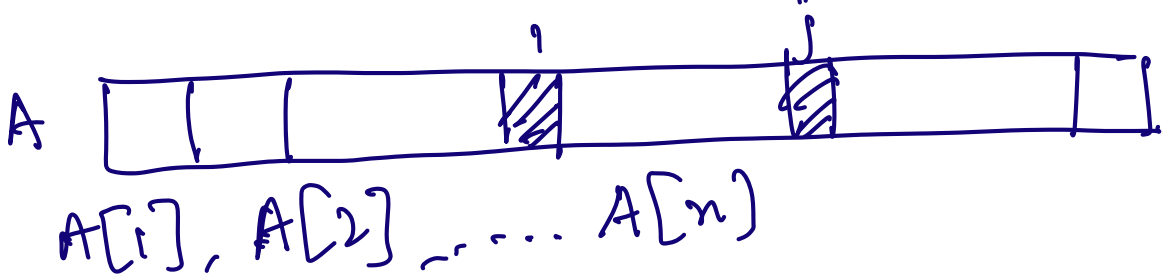
Comparing two rankings

- There are n songs.
- Your friend ranks these songs $1, \ldots, n$.
- You rank the songs $a_1, \ldots, a_n$.
- How many pairs of songs ranked by you are out of order?
- The pair (a_i, a_j) are out of order if $i < j$ but $a_i > a_j$.



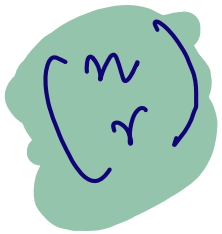
(6,5)
(3,2)
(6,4)
(6,2)

- If a pair (a_i, a_j) is out of order, then there is an inversion.



$a_i = A[i]$

Counting Inversion



$= {}^{n}C_{r}$

Problem definition:

- **Input:** An array A of n integers. The array A is indexed from $1, \ldots, n$.
- **Goal:** Compute $|\{(i,j) \mid i < j, A[i] > A[j]\}|$ (an *inversion* is an index pair (i,j) such that $i < j$ but $A[i] > A[j]$).

count the number of inversions

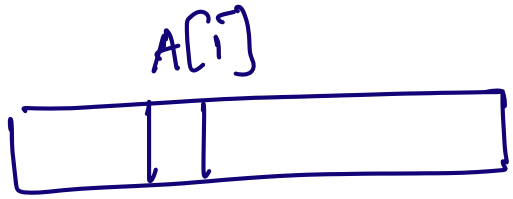
Some Questions:

- What is the number of inversions in a sorted array of n numbers? 0
- What can be the maximum number of inversions in an array of n numbers?

${}^{n}C_{2} = \binom{n}{2}$

An array in decreasing order

A naive algorithm



Input: An array A of n numbers.
Goal: Counting the number of inversions in A.

A naive algorithm

Input: An array A of n numbers.
Goal: Counting the number of inversions in A.

- Initialize a counter to 0.
- For every $(i, j) \in \binom{n}{2}$ such that $i < j$ check if $A[i] > A[j]$.
- Time taken is $\mathcal{O}(\binom{n}{2}) = \mathcal{O}(n^2)$.

A naive algorithm

Input: An array A of n numbers.
Goal: Counting the number of inversions in A.

- Initialize a counter to 0.
- For every $(i, j) \in \binom{n}{2}$ such that $i < j$ check if $A[i] > A[j]$.
- Time taken is $\mathcal{O}(\binom{n}{2}) = \mathcal{O}(n^2)$.

Questions:

- **Agenda:** How do we count the number of inversions in A in $\mathcal{O}(n \log n)$-time?
- **Question:** How can we output the set of all inversions in $\mathcal{O}(n \log n)$-time?

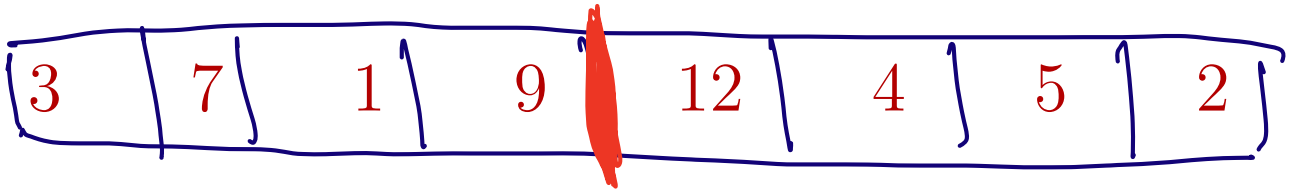
Not possible

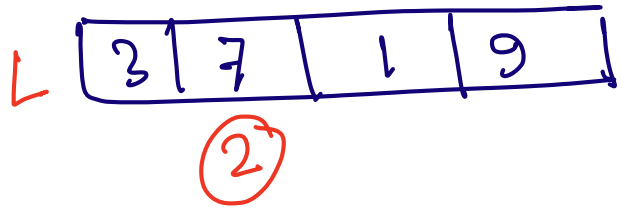
because there can be $O(n^2)$ inversions

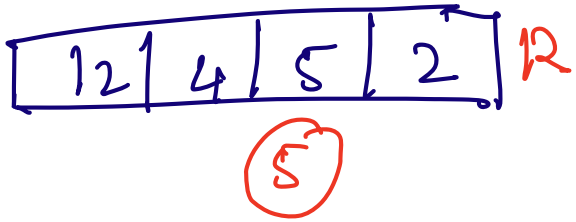
A recursive approach

Using Divide and Conquer

Example:







2+5 +9 ?

Split inversion: (i, j) s.t. $L[i] > R[j]$

$O(n)$ - time algorithm to count #split inversions.

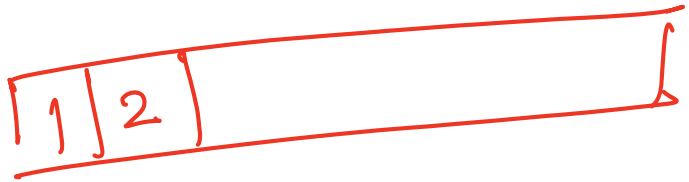
Counting Split Inversion

Input: There are two sorted arrays L and R.
Goal: Compute the number of split inversions between L and R. Meaning compute the number of pairs $(i, j) \in L \times R$ such that $L[i] > R[j]$.

- How can we do it in $\mathcal{O}(|L| + |R|)$-time when both are sorted arrays?

If $L[i] > R[j]$, then for all $k = \{i, i+1, \ldots |L|\}$

$L[k] > R[j]$



Some observation

Observation: If there is $i \in \{1, 2, \ldots, |L|\}$ and $j \in \{1, 2, \ldots, |R|\}$ and $L[i] > R[j]$, then for all $k = i, i+1, i+2, \ldots, |L|$ $L[k] > R[j]$.

It implies How many numbers of L are larger than $R[j]$? $\rightarrow$ $|L| - i + 1$

counter $\rightarrow$ increase by $|L| - i + 1$

Algorithm

```
Count-Inversion(A)
if |A| = 1 then
    Return (0, A);
end
else
    A_left ← first ⌈n/2⌉ elements of A;
    A_right ← the remaining ⌊n/2⌋ elements of A;
    (b_left, A_left) = Count-Inversion(A_left);
    (b_right, A_right) = Count-Inversion(A_right);
    (b_split, A) = Count-Split-Inversion(A_left, A_right);
    Return (b_left + b_right + b_split), and the sorted array A)
end
```

Merging Sorted Lists (revisit)

`Merge-Lists(`L, R`)`

Initialize $pt_\ell \leftarrow 1, pt_r \leftarrow 1$ *and* $i \leftarrow 1$;

Initialize an output array A *of size* $|L| + |R|$ *(all values to* ∞*);*

while $pt_\ell \leq |L|$ *and* $pt_r \leq |R|$ **do**

  if $L[pt_\ell] \leq R[pt_r]$ **then**

    $A[i] \leftarrow L[pt_\ell]$; $pt_\ell \leftarrow pt_\ell + 1$; $i \leftarrow i + 1$;

  end

  else

    $A[i] \leftarrow R[pt_r]$; $pt_r \leftarrow pt_r + 1$; $i \leftarrow i + 1$;

  end

end

if $pt_\ell > |L|$ **then**

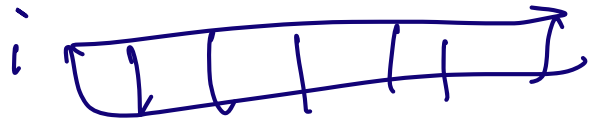
  Append the remaining elements of R *into* A*; Return* A*;*

end

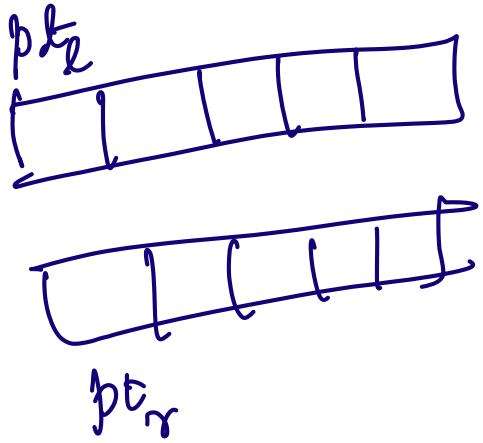
else

  Append the remaining elements of L *into* A*; Return* A*;*

end



Merge Sort



Split Inversion in sorted list

```
Count-Split-Inversion(L, R)
```

Initialize $pt_\ell \leftarrow 1, pt_r \leftarrow 1$, $i \leftarrow 1$, $count \leftarrow 0$;

Initialize an output array A *of size* $|L| + |R|$ *(all values to* ∞*);*

while $pt_\ell \leq |L|$ *and* $pt_r \leq |R|$ **do**

  if $L[pt_\ell] \leq R[pt_r]$ **then**

    $A[i] \leftarrow L[pt_\ell]$; $pt_\ell \leftarrow pt_\ell + 1$; $i \leftarrow i + 1$;

  end

  else

    $A[i] \leftarrow R[pt_r]$; $count \leftarrow count + |L| - pt_\ell + 1$; $pt_r \leftarrow pt_r + 1$;

    $i \leftarrow i + 1$;

  end

end

if $pt_\ell > |L|$ **then**

  Append the remaining elements of R *into* A*; Return* $(count, A)$*;*

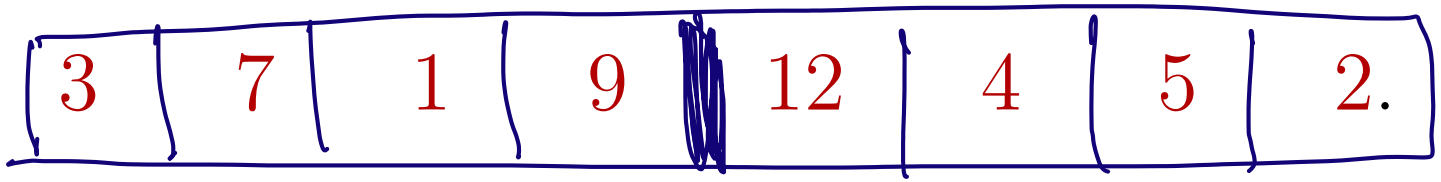
end

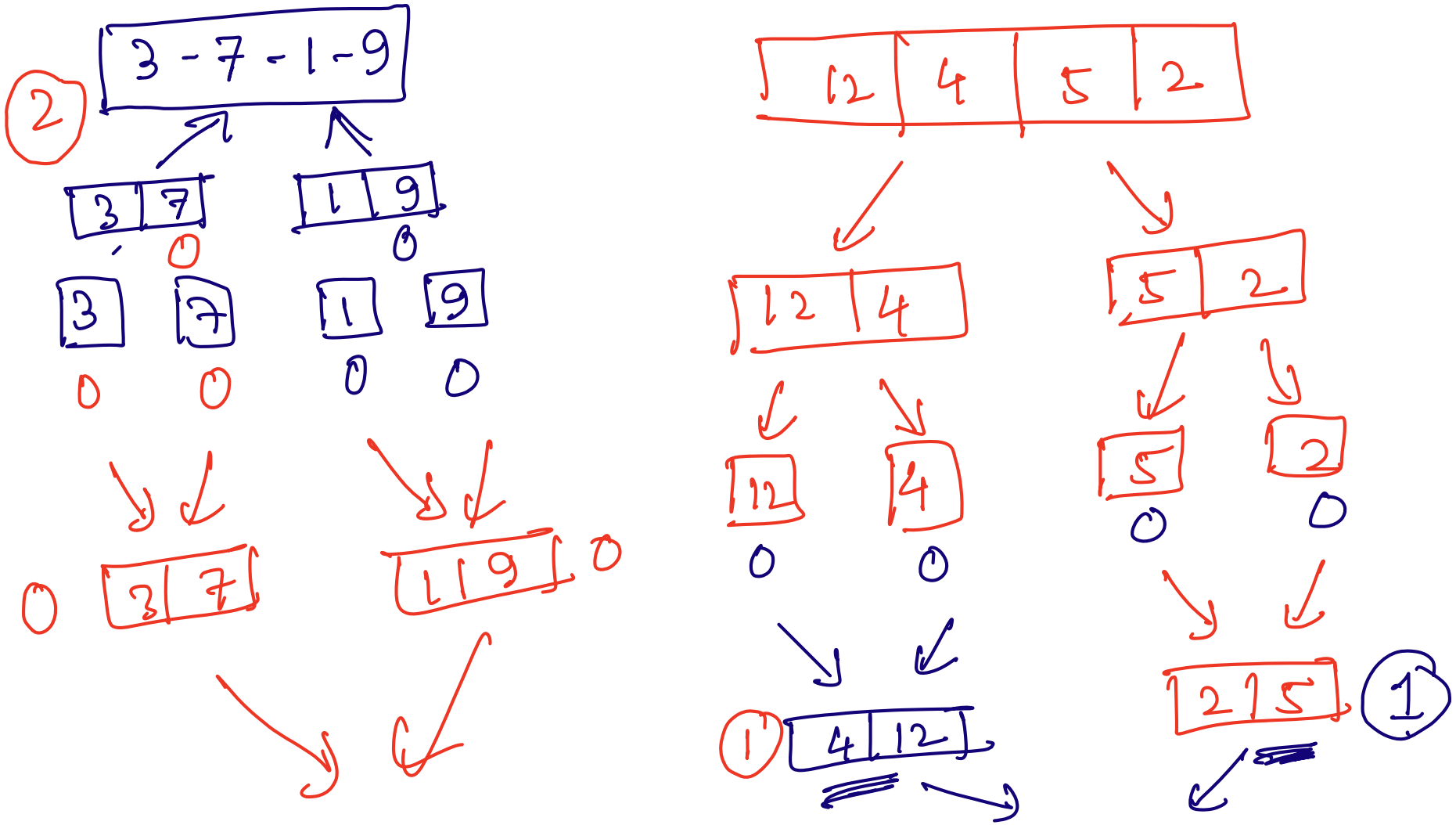
else

  Append the remaining elements of L *into* A*; Return* $(count, A)$*;*

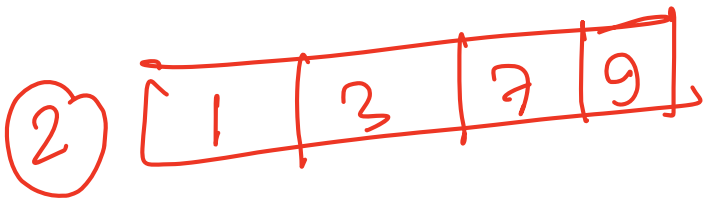
end

Split Inversion (example)

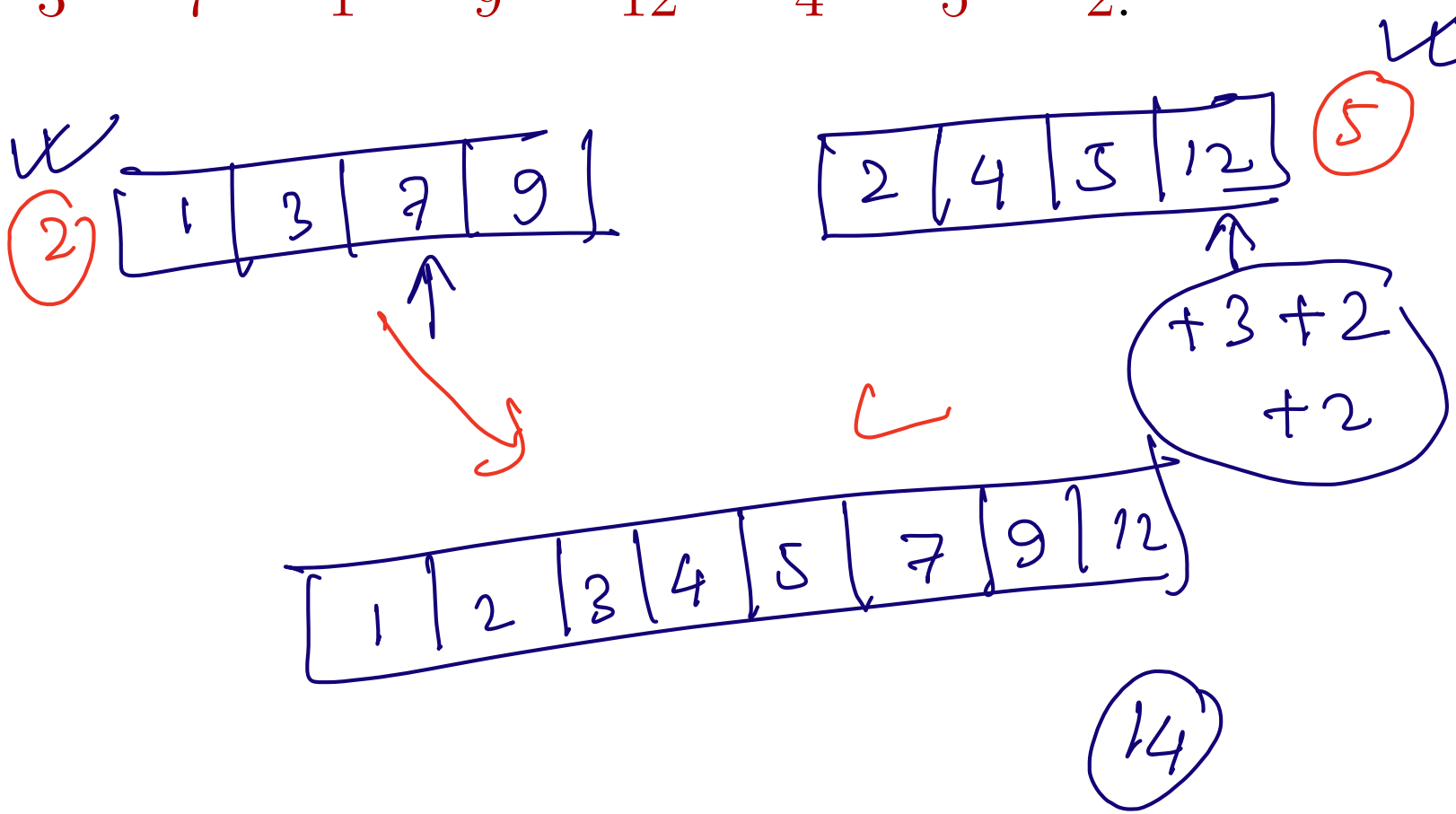




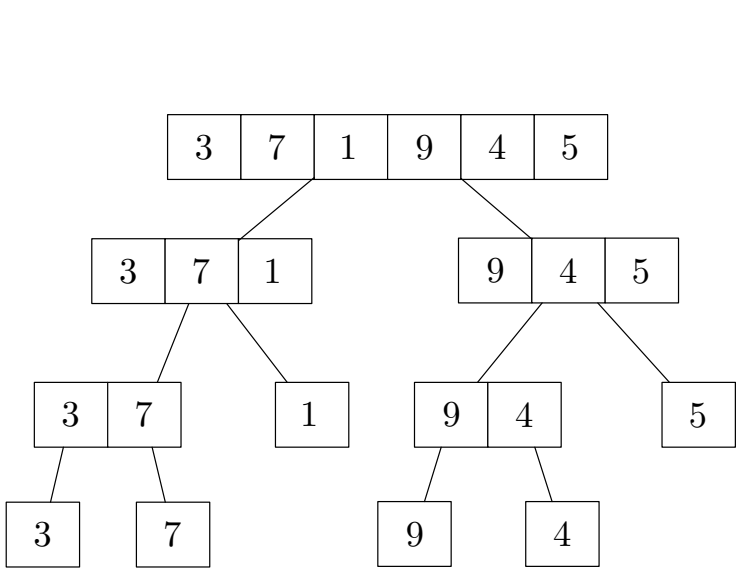
Example

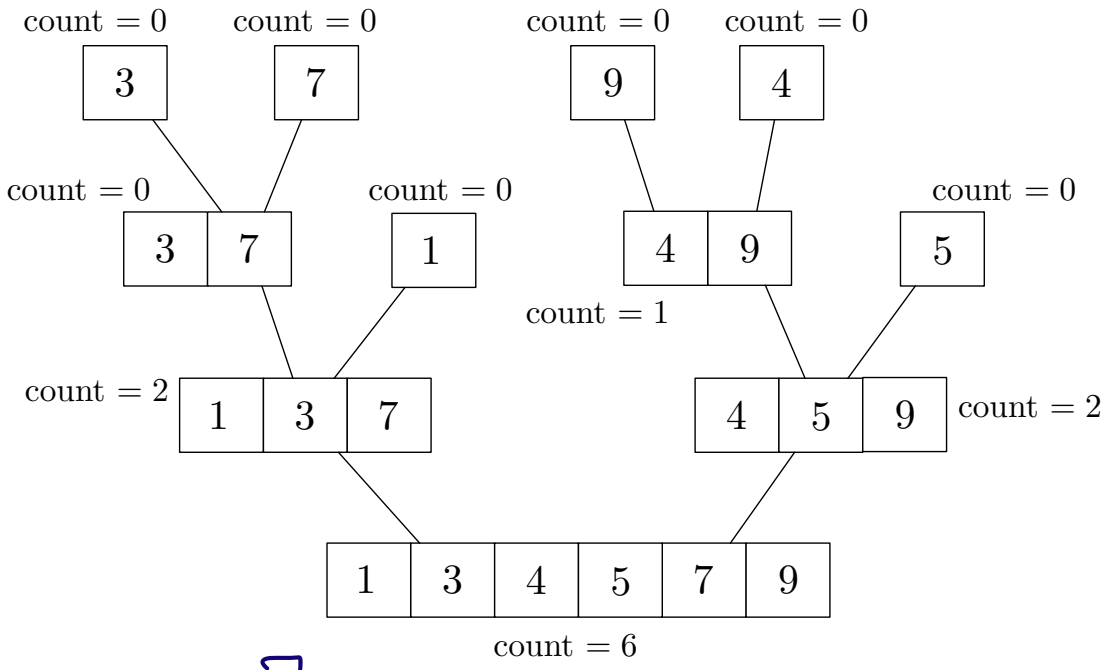


3 7 1 9 12 4 5 2.



Counting Inversion Example





$\lceil \log_2 n \rceil$

```
Count-Inversion(A)
if |A| = 1 then
    Return (0, A);
end
else
    A_left ← first ⌈n/2⌉ elements of A;
    A_right ← the remaining ⌊n/2⌋ elements of A;
    (b_left, A_left) = Count-Inversion(A_left);
    (b_right, A_right) = Count-Inversion(A_right);
    (b_split, A) = Count-Split-Inversion(A_left, A_right);
    Return (b_left + b_right + b_split), and the sorted array A)
end
```

```
Count-Inversion(A)
if |A| = 1 then
    Return (0, A);
end
else
    A_left ← first ⌈n/2⌉ elements of A;
    A_right ← the remaining ⌊n/2⌋ elements of A;
    (b_left, A_left) = Count-Inversion(A_left);
    (b_right, A_right) = Count-Inversion(A_right);
    (b_split, A) = Count-Split-Inversion(A_left, A_right);
    Return (b_left + b_right + b_split), and the sorted array A)
end
```

$$T(n) = T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + cn$$

Running time

What is the running time (or time complexity) of counting inversions?

- $T(n) = T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + cn.$

$= 2T(n/2) + cn$

$O(n \log_2 n)$

Why is it correct?

- Count-Split-Inversion algorithm is correct.

Proof (Sketch): Which proof method to use? (proof by induction or proof by contradiction?) Induction on the size of input n.

Base Case: $n = 1$. There is no inversion. Statement holds

Induction Hypothesis: The statement holds for all $n = 1, 2, \ldots, k-1$. (strong mathematical induction).

It means that the algorithm outputs correct answers for arrays of size $\frac{k}{2}$.

Consider $n = k$: There are two sorted arrays

L and R $|L|, |R| = \frac{k}{2}$.

Why the subroutine COUNT-SPLIT-INVERSION (L, R) is correct? $|L|, |R| = \frac{k}{2}$.

As L and R both are sorted, the algorithm has two pointers, one for L and other for R that move forward.

By observation (from earlier slide)

when $L[i] > R[j]$, then there are $(|L| - i + 1)$ elements that are larger than $R[j]$.

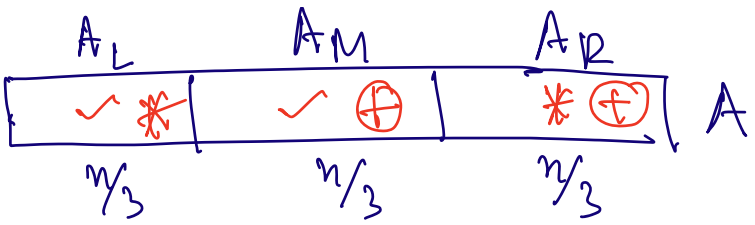
Hence, increasing the count by $|L| - i + 1$ is correct.

Finally, the total number of

inversions = number of inversions within L +
number of inversions within R +
the number of split inversions.

Reference

- Section 5.3 of Algorithm Design by J. Kleinberg and E. Tardos.



Sort and Count (A_L)

Sort and Count (A_M)

Sort and Count (A_R)

$3T(\frac{n}{3}) + cn$

Split (A_L, A_M) $\rightarrow O(2n/3)$

Split (A_L, A_R)

Split (A_M, A_R)

Does it work?

Think about it.